UNIQ ID: E0124092

Temporal Difference Prediction

TD Control Task : State Action Reward State Action (SARSA)

```
import gym
import random
import pandas as pd
```

```
Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality.
Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software and request that they upgrade.
See the migration guide at https://gymnasium.farama.org/introduction/migration_guide/ for additional information.
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a futu
  return datetime.utcnow().replace(tzinfo=utc)
```

```
#Now, we create the Frozen Lake environment using Gym:
env = gym.make('FrozenLake-v1')
```

```
/usr/local/lib/python3.13/dist-packages/gym/core.py:317: DeprecationWarning: WARN: Initializing wrapper in old step API which returns one bool instead of two. It is re
  deprecation(
/usr/local/lib/python3.13/dist-packages/gym/wrappers/step_api_compatibility.py:39: DeprecationWarning: WARN: Initializing environment in old step API which returns one
  deprecation(
```

```
#Initialize the discount factor γ, the learning rate α,
alpha = 0.85
gamma = 0.90

#and the epsilon value:
epsilon = 0.8
```

```
#Set the number of episodes and number of time steps in the episode:
num_episodes = 50000
num_timesteps = 1000
```

```
#define a random policy to choose random actions
def random_policy():
  return env.action_space.sample()
```

```
env.action_space
#Displays count of number of actions : left, right, top, down
```

```
Discrete(4)
```

TD Prediction

```
#Initialising the state values to zero.
#Create a empty dictionary : for storing the value of states (16 states, 4x4 grid)
v = {}
for s in range (env.observation_space.n):
  #Cannot make the observation space discrete, since it is 5x5 continous space
  v[s]=0.0
```

```
# Patch NumPy compatibility issues with older gym versions
import numpy as np
if not hasattr(np, 'bool8'):
    np.bool8 = np.bool_

#Compute the policy
#For each episode:
for i in range(num_episodes): #50000
#Initialize the state by resetting the environment:
    s = env.reset() #reset the frozenlake environment
#For each step in the episode:
    for t in range(num_timesteps): #1000
# Select an action according to the random policy
        a = random_policy()
#Perform the selected action and store the next state information:
        s_, r, done, _ = env.step(a)
#Compute the value of state as
        v[s] += alpha * (r + gamma * v[s_]-v[s]) # TD update rule
        #print(v[(s)])
        s = s_
        #a = a_
#If the current state is the terminal state, then break:
        if done: #Check for terminal state
            break
```

```
df = pd.DataFrame(list(v.items()), columns=['state', 'value'])
```

```
df
```

	state	value
0	0	2.429501e-05
1	1	4.734206e-06
2	2	2.840036e-04
3	3	3.209982e-07
4	4	1.588515e-07
5	5	0.000000e+00
6	6	2.150998e-03
7	7	0.000000e+00
8	8	1.521834e-01
9	9	3.573042e-01
10	10	1.870482e-02
11	11	0.000000e+00
12	12	0.000000e+00
13	13	6.421030e-02
14	14	9.824852e-01
15	15	0.000000e+00

```
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a futu
  return datetime.utcnow().replace(tzinfo=utc)
```

Next steps: Generate code with df | New interactive sheet

SARSA

```
# Now, let's define the epsilon-greedy policy. We generate a random number from
# the uniform distribution and if the random number is less than epsilon, we select the
# random action, else we select the best action that has the maximum Q value:
def epsilon_greedy(state, epsilon):
    if random.uniform(0,1) < epsilon:
        return env.action_space.sample()
    else:
        return max(list(range(env.action_space.n)), key = lambda x:Q[(state,x)])
```

```
#Let's define the dictionary for storing the Q value of the state-action pair and
#initialize the Q value of all the state-action pairs to 0.0:
Q = {}
for s in range(env.observation_space.n):
    for a in range(env.action_space.n):
        Q[(s,a)] = 0.0
```

```
#Compute the policy
#For each episode:
for i in range(num_episodes):
#Initialize the state by resetting the environment:
    s = env.reset()
#Select the action using the epsilon-greedy policy:
    a = epsilon_greedy(s,epsilon)
#For each step in the episode:
    for t in range(num_timesteps):
#Perform the selected action and store the next state information:
        s_, r, done, _ = env.step(a)
#Select the action a' in the next state s'using the epsilon-greedy policy:
        a_ = epsilon_greedy(s_,epsilon)
#Compute the Q value of the state-action pair as
        Q[(s,a)] += alpha * (r + gamma * Q[(s_,a_)]-Q[(s,a)])
        #print(Q[(s,a)])
# Update s=s' (update the next state s' to the current state s)
        s = s_
        a = a_
#If the current state is the terminal state, then break:
        if done:
            break
```

```
df = pd.DataFrame(list(Q.items()), columns=['state', 'value'])
```

```
df
```

	state	value
0	(0, 0)	0.015898
1	(0, 1)	0.004192
2	(0, 2)	0.004303
3	(0, 3)	0.012466
4	(1, 0)	0.000238
...	...	...
59	(14, 3)	0.144847
60	(15, 0)	0.000000
61	(15, 1)	0.000000
62	(15, 2)	0.000000
63	(15, 3)	0.000000

64 rows × 2 columns

```
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a futu
  return datetime.utcnow().replace(tzinfo=utc)
```

Next steps: Generate code with df | New interactive sheet

Q Learning

```
import numpy as np
```

```
#Compute the policy
#For each episode:
for i in range(num_episodes):
#Initialize the state by resetting the environment:
    s = env.reset()
#For each step in the episode:
    for t in range(num_timesteps):
#Select the action using the epsilon-greedy policy:
        a = epsilon_greedy(s,epsilon)
#Perform the selected action and store the next state information:
        s_, r, done, _ = env.step(a)
#Select the action a' that has the maximum Q value in the next state s':
        a_ = np.argmax([Q[(s_,a)] for a in range(env.action_space.n)])
#Compute the Q value of the state-action pair as
        Q[(s,a)] += alpha * (r + gamma * Q[(s_,a_)]-Q[(s,a)])
        #print(Q[(s,a)])
# Update s=s' (update the next state s' to the current state s)
        s = s_
        #a = a_
#If the current state is the terminal state, then break:
        if done:
            break
```

```
df = pd.DataFrame(list(Q.items()), columns=['state', 'value'])
```

```
df
```

	state	value
0	(0, 0)	0.239305
1	(0, 1)	0.202921
2	(0, 2)	0.205508
3	(0, 3)	0.203564
4	(1, 0)	0.186644
...	...	...
59	(14, 3)	0.740639
60	(15, 0)	0.000000
61	(15, 1)	0.000000